

SPEC-1-Supporters Bus Attendance App

Background

A football supporters group needs a mobile app for iOS and Android to manage attendance for supporters buses travelling to matches and related events. Members should receive a push notification for each game/event and confirm whether they will take a seat. A single app user may represent multiple members, such as children, elderly relatives, or people without smartphones.

Users must also be able to request additional non-member seats for guests. Admins need a clear attendance view per game/event showing confirmed attendees, declined members, and members who have not responded. Admins must be able to set a maximum capacity for each bus/event, send reminder notifications, add additional events, and receive alerts as capacity approaches, reaches, or exceeds the configured limit.

The MVP should support older iOS and Android versions while keeping development practical for contractors building a single cross-platform product.

Requirements

Must Have

- Support iOS and Android from a single codebase where possible.
- Prioritise maximum practical backwards compatibility for older smartphones.
- Maintain a managed member list of approximately 60 members.
- Store a phone number against each member as the primary real-world identifier.
- Allow one logged-in app user/phone number to represent one or more member records.
- Allow admins to create, edit, deactivate, approve, and link member records to a user account/phone number.
- Allow representation to be configured by admins and also requested during initial user sign-up.
- During initial sign-up, allow a user to create or claim their own member profile and add one or more child/dependent member profiles they represent.
- Example: one father user account can represent himself plus two child member records.
- Allow admins to review and approve member/dependent representation requests submitted during sign-up.
- Allow admins to create games/events requiring bus attendance confirmation.
- Allow each event to have an editable text title, for example the match name, trip name, or special event name.
- Allow optional event details including route/stops, departure time, destination/venue, and general notes.
- Allow admins to configure multiple travel stops per event, typically including:
 - outbound pickup stops before departure
 - event drop-off location
 - event pickup location for the return journey
 - return drop-off stops after the event

- Allow admins and helpers to update travel stops on the day when plans change.
- Store each stop with a name, type, order, optional scheduled time, map coordinates, and notes.
- Allow each attending member to select their outbound pickup stop and return drop-off stop where relevant.
- Allow each named guest request to select only the initial pickup stop from the admin-created list of outbound pickup stops.
- Do not require guests to select event drop-off, event pickup, or return drop-off stops in the MVP.
- Allow users to open stop locations in Apple Maps or Google Maps from the app.
- Allow admins and helpers to update the parked-bus location after arrival using a map pin and additional notes.
- Allow admins and helpers to send operational event update notifications, such as pickup delays, route/stop changes, parked-bus location updates, or event notes.
- Send operational event update notifications only to users marked as attending, plus relevant admins/helpers.
- When route/stop details change, notify all attending users rather than trying to target only affected stops in the MVP.
- Allow users to open the parked-bus location in Apple Maps or Google Maps from the app.
- Allow admins to set a maximum capacity for each game/event.
- Send push notifications to relevant users when a game/event requires confirmation.
- Allow users to confirm whether represented members are attending or not attending.
- Treat normal member attendance confirmations as immediately confirmed without admin approval.
- Allow users to request additional non-member/guest seats per event by entering guest names.
- Show admins per-event lists grouped by:
 - attending
 - not attending
 - still to confirm
- Show admins total requested capacity, including confirmed member seats, pending guest seat requests, and approved guest seats.
- Alert admins when approved capacity is close to, equal to, or above the configured maximum.
- Alert admins when pending guest requests could cause the event to exceed capacity if approved.
- Allow admins to send reminder push notifications to users who have not confirmed.
- Allow admins to send different attendance reminder messages for:
 - users who have not responded at all
 - users with pending guest seat requests
- Keep attendance reminders separate from operational event update notifications.
- Provide an admin-only area for managing members, events, confirmations, reminders, and capacity.
- Provide a helper role that can update operational event details such as parked-bus location and notes, without full admin access.
- Allow admins to assign multiple helpers per event.

Should Have

- Support login via phone number using one-time passcode, avoiding email/password complexity for older users.
- Do not require each member record to have its own app login or device.
- Allow a user account to represent the user's own member record as well as one or more child/dependent member records.
- Allow admins to set an optional confirmation cutoff date/time per event.

- Allow users to update member attendance and guest requests until the event cutoff, when one is configured.
- Allow admins to approve or reject named non-member guest seat requests.
- Prevent users from submitting or changing guest requests after the cutoff unless an admin manually overrides the event response.
- Allow admins to manually override a member/event response if someone confirms outside the app.
- Keep a simple audit trail of admin changes and notification reminders.
- Provide clear large-button mobile screens suitable for less technical users.
- Show users only current and upcoming events in the main app experience.
- Allow admins to view past events.
- Support SMS fallback or manual admin entry for members with phones too old for the app or without smartphones.

Could Have

- Waitlist handling when the bus is over capacity.
- General notes field for member attendance or guest requests.
- Export attendance lists to CSV/PDF for printing or sharing.
- Future support for multiple buses per event.
- In-app announcements for pickup times, delays, or changes.
- Optional event metadata such as opponent, venue, route stops, departure time, stop map pins, parked-bus map pin, parked-bus notes, and general notes.
- Allow admins to query historical attendance by member, event, and date range.
- Member notes such as accessibility needs, child/elderly status, or emergency contact.

Won't Have in MVP

- Multiple buses per event. MVP supports one bus per event, but the data model should not block adding multiple buses later.
- Online payments for seats.
- Public event discovery or registration by unknown users.
- Complex ticketing system.
- Live bus tracking.
- Social feed or chat functionality.

Non-Functional Requirements

- The app must use a simple, large-button interface suitable for older or less technical users.
- The app should continue to show previously loaded current/upcoming event data when offline, where Firebase local cache allows it.
- Backend permission checks must prevent users from responding for members they are not linked to.
- Admin/helper actions must be auditable.
- Push notifications must be recorded in the database so delivery issues can be investigated.
- The app should avoid continuous background location tracking. Location should only be captured when an admin/helper explicitly pins a stop or parked-bus location.
- Personal data should be limited to what is needed for attendance management: names, phone numbers, member links, attendance records, guest names, and operational notes.

- The app should support the latest two major OS versions well, while still targeting the agreed older baseline of iOS 13+ and Android 7.0/API 24+ where technically supportable.

Assumptions

- The supporters organisation has approximately 60 active members at launch.
- Admins will maintain the authoritative member list.
- A member is not the same as a login account.
- A phone number can be associated with more than one member record.
- A user account can represent multiple members only when configured by an admin.
- The MVP supports one bus per event.
- Guest seats require admin approval before they count as approved capacity.
- Operational notifications are only sent to attending users, admins, and assigned helpers.
- Admins remain responsible for resolving edge cases such as late confirmations, changed phone numbers, or people who cannot use the app.

Method

Recommended MVP Architecture

Use a supportable cross-platform mobile architecture:

- **Mobile app:** Flutter/Dart for iOS and Android from one codebase.
- **Minimum supported versions:** iOS 13+ and Android 7.0/API 24+ for the main app.
- **Fallback for older/non-smartphone users:** admin manual entry, phone/SMS outside the app, and admin overrides.
- **Backend:** Firebase.
- **Authentication:** Firebase Authentication phone number login with one-time SMS code.
- **Database:** Cloud Firestore.
- **Push notifications:** Firebase Cloud Messaging.
- **Server-side rules and automation:** Cloud Functions for Firebase.
- **Maps:** store coordinates in Firestore; open locations in Apple Maps/Google Maps using an external maps launcher. For helpers, use device GPS to pin the parked-bus location.

This avoids maintaining separate native iOS and Android apps while keeping the product simple enough for a contractor team to build and support.

Similar Product Analysis

Existing apps such as **Spond**, **TeamSnap**, and **Stack Team App** already support parts of this problem space: team events, availability, rosters, attendance reminders, and push notifications. However, the proposed app needs custom behaviour that generic club apps do not handle cleanly:

- one phone user representing multiple member records
- named non-member guest seat requests
- admin approval for guests
- bus capacity warnings
- configurable outbound and return route stops

- parked-bus pin updates on the day
- attendance history queryable by member

The MVP should therefore follow the proven event/availability pattern from these apps, but use a custom data model focused on bus seat management.

Core Domain Model

User Account

A login identity. Authenticated by phone number + one-time code.

Member

A managed supporter/member record. A member may or may not have their own phone.

Member Representation Link

Link allowing a user account to respond for one or more members, including their own member record and any child/dependent members. Links may be created by admins or requested during sign-up and approved by admins.

Event

A football match, trip, or related supporter event requiring bus confirmation.

Route Stop

A configurable stop for outbound pickup, event drop-off, event pickup, or return drop-off.

Member Response

One attendance response for one member on one event.

Guest Request

Named non-member seat request submitted by a user and approved/rejected by admin.

Notification

Recorded push notification event for audit and troubleshooting.

Roles and Permissions

Role	Permissions
User	Confirm attendance for linked members, request named guest seats, choose allowed pickup/drop-off stops, view current/upcoming events, open map locations.
Helper	User permissions plus update parked-bus location, route/stop notes, and send operational event updates for assigned events.

Role	Permissions
Admin	Full access to members, users, links, events, helpers, route stops, responses, guest approvals, reminders, capacity, overrides, and attendance history.

Admins can assign multiple helpers to an event. Helper permissions are scoped to assigned events.

High-Level Component Diagram

```

@startuml
skinparam componentStyle rectangle

actor MemberUser as "User / Member Representative"
actor Helper
actor Admin

component "Flutter Mobile App"
  iOS + Android as App
  component "Firebase Auth"
    Phone OTP as Auth
  component "Cloud Firestore" as DB
  component "Cloud Functions" as Functions
  component "Firebase Cloud Messaging" as FCM
  component "External Maps"
    Apple Maps / Google Maps as Maps
  end
end

MemberUser --> App
Helper --> App
Admin --> App

App --> Auth : phone login
App --> DB : read/write allowed data
App --> Functions : privileged actions
Functions --> DB : validate + update
Functions --> FCM : send push notifications
App --> Maps : open coordinates

DB --> Functions : triggers for capacity/status changes
@enduml

```

Firestore Schema

```
users/{userId}
```

Created after successful phone OTP login. `userId` is the Firebase Auth UID.

```
{
  "phoneE164": "+447700900123",
  "displayName": "John Smith",
  "roles": ["user"],
  "isActive": true,
  "createdAt": "timestamp",
  "updatedAt": "timestamp",
  "lastLoginAt": "timestamp"
}
```

Subcollection: `users/{userId}/fcmTokens/{tokenId}`

```
{
  "token": "fcm-token",
  "platform": "ios | android",
  "createdAt": "timestamp",
  "lastSeenAt": "timestamp"
}
```

`members/{memberId}`

A member is a supporter record, not necessarily a login account.

```
{
  "firstName": "John",
  "lastName": "Smith",
  "displayName": "John Smith",
  "primaryPhoneE164": "+447700900123",
  "memberNumber": "optional-club-reference",
  "status": "pending | active | rejected | inactive",
  "generalNotes": "optional",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

Phone numbers can repeat across members when one person represents children or elderly relatives.

`memberUserLinks/{linkId}`

Representation link between a user account and a member record. It may be created directly by an admin or requested during sign-up and then approved by an admin.

```
{
  "userId": "firebase-auth-uid",
  "memberId": "member-id",
  "status": "active | inactive",
  "relationshipToUser": "self | child | dependent | other",
  "requestedDuringSignup": true,
  "createdByAdminId": "admin-user-id-or-null",
  "approvedByAdminId": "admin-user-id-or-null",
  "approvedAt": "timestamp-or-null",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

Indexes:

```
memberUserLinks by userId + status
memberUserLinks by memberId + status
memberUserLinks by status + createdAt
```

Signup-created links should normally start as `pending` until an admin approves them. If the organisation wants a faster but less controlled onboarding process, links can be auto-approved only when the phone number exactly matches an existing trusted member record.

`events/{eventId}`

One bus per event in MVP.

```
{
  "title": "Rangers v Example FC",
  "eventDate": "timestamp",
  "status": "draft | open | closed | completed | cancelled",
  "capacityMax": 53,
  "capacityNearThresholdPercent": 90,
  "cutoffAt": "timestamp-or-null",
  "destinationName": "Example Stadium",
  "generalNotes": "optional",
  "parkedBusLocation": {
    "lat": 55.860916,
    "lng": -4.251433,
    "label": "Car Park B",
    "notes": "Near north gate",
    "updatedByUserId": "helper-or-admin-id",
    "updatedAt": "timestamp"
  },
}
```



```

"capacityStatus": "under | near | at | over",
"pendingGuestRisk": false,
"lastCapacityAlertSentAt": "timestamp-or-null",
"createdByAdminId": "admin-user-id",
"createdAt": "timestamp",
"updatedAt": "timestamp"
}

```

events/{eventId}/stops/{stopId}

```

{
  "name": "Main Street Clubhouse",
  "type": "outbound_pickup | event_dropoff | event_pickup | return_dropoff",
  "sequence": 1,
  "scheduledAt": "timestamp-or-null",
  "location": {
    "lat": 55.864237,
    "lng": -4.251806,
    "address": "Optional address text"
  },
  "notes": "Meet outside clubhouse",
  "isActive": true,
  "updatedByUserId": "admin-or-helper-id",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}

```

Rules:

- Members select from active `outbound_pickup` and `return_dropoff` stops.
- Guests select only from active `outbound_pickup` stops.
- Admins/helpers can update stops on the day.
- Stop changes trigger operational notifications to attending users only.

events/{eventId}/helpers/{userId}

```

{
  "userId": "helper-user-id",
  "assignedByAdminId": "admin-user-id",
  "createdAt": "timestamp"
}

```

events/{eventId}/memberResponses/{memberId}

One document per event/member.

```
{
  "memberId": "member-id",
  "respondingUserId": "user-id-who-submitted",
  "status": "attending | not_attending",
  "outboundPickupStopId": "stop-id-or-null",
  "returnDropoffStopId": "stop-id-or-null",
  "generalNotes": "optional",
  "isAdminOverride": false,
  "overriddenByAdminId": "admin-id-or-null",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

Validation:

- A normal user can only create/update responses for members linked to their user account.
- A normal user can only update before `cutoffAt` when a cutoff exists.
- Admins can override after cutoff.
- Member attendance does not require admin approval.

`events/{eventId}/guestRequests/{guestRequestId}`

One document per named guest.

```
{
  "guestName": "Jane Guest",
  "requestedByUserId": "user-id",
  "linkedMemberId": "member-id-or-null",
  "initialPickupStopId": "outbound-pickup-stop-id",
  "status": "pending | approved | rejected | cancelled",
  "decisionByAdminId": "admin-id-or-null",
  "decisionAt": "timestamp-or-null",
  "generalNotes": "optional",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

Validation:

- User must enter guest name.
- User must choose an initial outbound pickup stop.
- Guest starts as `pending`.
- Only admins can approve or reject.
- Approved guests count against confirmed capacity.
- Pending guests are shown as potential capacity risk.

notifications/{notificationId}

```
{
  "eventId": "event-id-or-null",
  "type": "attendance_request | attendance_reminder | pending_guest_reminder |
guest_approved | guest_rejected | capacity_alert | operational_update",
  "title": "Bus attendance reminder",
  "body": "Please confirm your seats for Saturday.",
  "targetUserIds": ["user-id-1", "user-id-2"],
  "sentByUserId": "admin-or-helper-or-system",
  "status": "queued | sent | partial_failure | failed",
  "createdAt": "timestamp",
  "sentAt": "timestamp-or-null"
}
```

Capacity Calculation

Capacity is calculated from confirmed member attendance and approved guests.

```
confirmedMemberSeats = count(memberResponses where status = attending)
approvedGuestSeats = count(guestRequests where status = approved)
pendingGuestSeats = count(guestRequests where status = pending)

approvedSeats = confirmedMemberSeats + approvedGuestSeats
potentialSeats = approvedSeats + pendingGuestSeats
nearLimit = ceil(capacityMax * capacityNearThresholdPercent / 100)

if approvedSeats > capacityMax: capacityStatus = over
else if approvedSeats == capacityMax: capacityStatus = at
else if approvedSeats >= nearLimit: capacityStatus = near
else: capacityStatus = under

pendingGuestRisk = potentialSeats > capacityMax
```

A Cloud Function recalculates these values when:

- a member response is created/updated/deleted
- a guest request is created/updated/deleted
- event capacity is changed

Capacity alert rules:

- Notify admins when status changes to `near`, `at`, or `over`.
- Notify admins when pending guests could push the event over capacity.
- Do not repeatedly send the same capacity alert unless the status changes again.

Attendance Workflow

```
@startuml
start
:Admin creates event;
:Admin configures capacity, cutoff, route stops;
:Admin opens event and sends attendance request;
:Linked users receive push notification;
:User opens event;
:App loads members linked to user;

repeat
:User marks each represented member as attending or not attending;
if (Attending?) then (yes)
:Select outbound pickup stop;
:Select return drop-off stop;
endif
repeat while (More linked members?) is (yes)

if (User needs guest seats?) then (yes)
repeat
:Enter guest name;
:Select initial pickup stop only;
:Submit guest request as pending;
repeat while (More guests?) is (yes)
endif
endif

:Member responses are confirmed immediately;
:Guest requests wait for admin approval;
:Capacity recalculated;
stop
@enduml
```

Guest Approval Workflow

```
@startuml
start
:Guest request submitted;
:Status = pending;
:Admin views pending guest list;
:Show approved capacity and potential capacity;
if (Approve?) then (yes)
:Set status = approved;
:Notify requesting user;
:Recalculate capacity;
else (no)
```

```
:Set status = rejected;  
:Notify requesting user;  
endif  
stop  
@enduml
```

Reminder and Notification Logic

Attendance Request

When an event is opened, admins can send an attendance request to all users who represent at least one active member.

Attendance Reminder

Target users where at least one linked active member does not yet have a response for the event.

```
for each active user:  
  linkedMembers = active memberUserLinks for user  
  missingResponses = linkedMembers without event memberResponse  
  if missingResponses is not empty:  
    include user in reminder target list
```

Pending Guest Reminder

Target users with one or more pending guest requests for the event.

Operational Event Update

Can be sent by admins or assigned helpers. Examples:

- pickup time changed
- stop location changed
- bus parked location added
- bus delay note

Target only:

- users with at least one attending member response
- admins
- assigned helpers

Cutoff Enforcement

User change allowed when:

event.cutoffAt is null
OR currentTime < event.cutoffAt

After cutoff:

users cannot change member responses
users cannot add/change/cancel guest requests
admins can still override responses and guest decisions

Cloud Functions and Firestore Security Rules should both enforce cutoff rules. The app UI should hide blocked actions, but backend rules must be the source of truth.

Route and Maps Method

For each event, admins create a route plan from ordered stops:

Outbound:

1. outbound_pickup
2. outbound_pickup
3. outbound_pickup
4. event_dropoff

Return:

5. event_pickup
6. return_dropoff
7. return_dropoff
8. return_dropoff

Users can open any stop location in Apple Maps or Google Maps. To maximise older-device compatibility, the MVP should avoid relying on heavy in-app navigation. The app stores coordinates and launches the user's installed map app.

Parked-bus location:

- Admin/helper opens event.
- Taps **Set parked bus location**.
- App asks for location permission.
- App captures current GPS coordinates.
- Admin/helper adds optional label and notes.
- App updates `events/{eventId}.parkedBusLocation`.
- App sends operational notification to attending users.

Optional enhancement:

- Add Google Places/Geocoding for admin address search when creating stops.
- Keep API keys server-side where possible and restrict keys by platform/app bundle.

Security Rules Strategy

Use Firebase Auth custom claims and Firestore rules:

```
admin:
  can manage all records

helper:
  can read event data
  can update stops, parked-bus location, and operational notes only for assigned
  events
  cannot approve guests
  cannot manage members
  cannot send attendance reminders

user:
  can read own user profile
  can read members linked to their user ID
  can read current/upcoming events
  can create/update own member responses before cutoff
  can create/cancel own guest requests before cutoff
  cannot approve guest requests
  cannot change event capacity
```

Privileged operations such as sending notifications, approving guests, recalculating capacity, assigning helpers, and importing members should go through Cloud Functions to keep business rules consistent.

MVP Screens

User Screens

- Phone login screen
- Current/upcoming events list
- Event detail screen
- Linked member attendance form
- Guest request form
- Route stops screen
- Open in maps action
- Parked-bus location screen

Helper Screens

- Assigned events list
- Event route/stop update screen
- Set parked-bus location screen
- Send operational update screen

Admin Screens

- Admin dashboard
- Member list and member editor
- User/member link management
- Event list: upcoming and past
- Event create/edit screen
- Route stop editor
- Attendance board: attending / not attending / still to confirm
- Guest approval queue
- Capacity status panel
- Send attendance reminder screen
- Assign helpers screen
- Historical attendance query screen

Attendance History Queries

For MVP, Firestore queries should support:

Events attended by member:

```
Query collectionGroup('memberResponses')
  where memberId == selectedMemberId
  where status == attending
  order by eventDate via duplicated eventDate field or joined event lookup
```

Members attending event:

```
Query events/{eventId}/memberResponses
  where status == attending
```

Guest requests by event:

```
Query events/{eventId}/guestRequests
  order by status, guestName
```

Attendance by date range:

```
Query events where eventDate between start/end
Then load memberResponses for selected events
```

To make history easier, duplicate these fields onto `memberResponses`:


```
{
  "eventId": "event-id",
  "eventTitle": "Rangers v Example FC",
  "eventDate": "timestamp",
  "memberDisplayName": "John Smith"
}
```

This is acceptable denormalisation for Firestore and keeps admin reporting simple for a small 60-member organisation.

Contractor Build Boundaries

The MVP should be delivered as:

- one Flutter mobile app with role-based screens
- one Firebase project
- Firestore database and indexes
- Firebase Auth phone login
- Firebase Cloud Messaging push setup for iOS and Android
- Cloud Functions for notifications, capacity recalculation, guest approvals, and admin actions
- seed/import script for the initial member list
- basic audit trail for admin actions and notifications
- internal testing distribution via TestFlight and Google Play internal testing

Implementation

1. Project Setup

Create one source repository for the Flutter app and Firebase backend.

Suggested structure:

```
supporters-bus-app/
  app/                # Flutter mobile app
  functions/          # Firebase Cloud Functions
  firestore/          # rules, indexes, seed data
  scripts/            # member import and admin utilities
  docs/               # setup/runbook documentation
```

Create separate Firebase projects for:

```
supporters-bus-dev
supporters-bus-prod
```

Enable these Firebase services:

- Firebase Authentication with phone number sign-in
- Cloud Firestore
- Firebase Cloud Messaging
- Cloud Functions
- Firebase Crashlytics
- Firebase App Check, after the first MVP test build is stable

2. Mobile App Foundation

Create a Flutter application under `app/`.

Recommended app layers:

```
lib/  
  main.dart  
  app.dart  
  core/  
    auth/  
    routing/  
    permissions/  
    errors/  
    widgets/  
  features/  
    login/  
    events/  
    attendance/  
    guests/  
    route_stops/  
    helper/  
    admin/  
    history/  
  data/  
    models/  
    repositories/  
    firebase/
```

Recommended Flutter packages:

```
dependencies:  
  firebase_core: latest-compatible  
  firebase_auth: latest-compatible  
  cloud_firestore: latest-compatible  
  firebase_messaging: latest-compatible
```

```
firebase_crashlytics: latest-compatible
firebase_app_check: latest-compatible
cloud_functions: latest-compatible
geolocator: latest-compatible
maps_launcher: latest-compatible
intl: latest-compatible
collection: latest-compatible
```

Use exact versions when implementation starts and lock them in `pubspec.lock`.

3. Authentication and Initial Signup Flow

Implement phone number login and first-time member setup:

1. User enters mobile number.
2. App normalises to E.164 format, for example `+447700900123`.
3. Firebase sends one-time SMS code.
4. User enters code.
5. App signs in and obtains Firebase Auth UID.
6. App loads `users/{userId}`.
7. If user profile does not exist, create a minimal user profile with `roles: ["user"]` and `isActive: true`.
8. If this is the user's first signup, ask whether they are registering:
 9. only themselves
 10. themselves plus child/dependent members
 11. child/dependent members only, if the adult is not a club member
12. Create or request member records for the user and any child/dependent members entered during signup.
13. Create `memberUserLinks` for those member records with `status: pending`, unless the organisation chooses auto-approval for exact phone-number matches.
14. App checks approved linked members via `memberUserLinks` where `status == active`.

Important behaviour:

- A user may log in successfully but have no approved linked members yet.
- If no approved linked members exist, show a friendly message: **Your account is active, but your member links are awaiting admin approval.**
- Admins can link members to users by phone number.
- Admins can approve, reject, or edit signup-created member/dependent links.

4. Admin Member Management

Build admin screens for:

- member list
- create/edit member
- activate/deactivate member
- search by name or phone
- link member to user account
- unlink/deactivate member-user link

Initial import process:

1. Admin prepares a CSV file.
2. Contractor runs `scripts/import_members.ts` or equivalent.
3. Script creates `members` records.
4. Script optionally creates `memberUserLinks` for known phone/member relationships.

Suggested CSV format:

```
firstName,lastName,displayName,primaryPhoneE164,memberNumber,generalNotes
John,Smith,John Smith,+447700900123,001,
Child,Smith,Child Smith,+447700900123,002,
```

5. Event Creation and Route Setup

Build admin event create/edit flow:

Required fields:

```
title
eventDate
capacityMax
status
```

Optional fields:

```
cutoffAt
destinationName
generalNotes
capacityNearThresholdPercent, default 90
```

Route setup screen:

- Add stop
- Edit stop
- Reorder stops
- Deactivate stop
- Set stop type
- Add scheduled time
- Add map coordinates
- Add notes

Stop types:

```
outbound_pickup
event_dropoff
event_pickup
return_dropoff
```

MVP route validation:

- At least one active `outbound_pickup` stop is required before opening attendance.
- At least one active `return_dropoff` stop is recommended but not mandatory.
- Event drop-off and event pickup are recommended for trips but should remain optional.

6. User Attendance Flow

User opens current/upcoming event.

App loads:

```
linked active members
current member responses for the event
active route stops
current guest requests made by the user
cutoff status
capacity display message
```

For each linked member:

- show member name
- choose attending / not attending / not answered
- if attending:
 - choose outbound pickup stop
 - choose return drop-off stop, if return stops exist
- optional general notes

Save each member response to:

```
events/{eventId}/memberResponses/{memberId}
```

Backend validation must ensure:

- user can only respond for linked members
- user cannot update after cutoff
- selected pickup stop is active and has type `outbound_pickup`
- selected return stop is active and has type `return_dropoff`

7. Guest Request Flow

From the event detail screen, user can add named guests.

Guest form fields:

```
guestName, required  
initialPickupStopId, required, must be an active outbound_pickup stop  
generalNotes, optional  
linkedMemberId, optional
```

On submit:

- create guest request with `status: pending`
- do not count it as approved capacity yet
- show the user that the request is awaiting admin approval

Users may cancel or edit pending guest requests before cutoff. Approved or rejected guest requests should not be user-editable; admins can override if needed.

8. Admin Attendance Board

Build an event attendance board with tabs or grouped panels:

```
Attending  
Not attending  
Still to confirm  
Pending guest requests  
Approved guests  
Rejected guests  
Capacity  
Route selections
```

Attendance board should show:

- member name
- representing user/phone number
- response status
- outbound pickup stop
- return drop-off stop
- notes
- last updated time
- admin override marker

Guest board should show:

- guest name
- requested by user
- linked member, if provided
- initial pickup stop
- status
- notes
- approve/reject actions

Capacity panel:

```
Member seats confirmed
Approved guest seats
Approved total
Pending guest seats
Potential total if all pending are approved
Capacity maximum
Capacity status: under / near / at / over
```

9. Cloud Functions

Implement Cloud Functions using TypeScript.

Recommended functions:

```
onMemberResponseWrite
  Trigger: events/{eventId}/memberResponses/{memberId} write
  Action: recalculate event capacity

onGuestRequestWrite
  Trigger: events/{eventId}/guestRequests/{guestRequestId} write
  Action: recalculate event capacity and pending guest risk

approveGuestRequest
```

Callable HTTPS function
Role: admin only
Action: approve guest, recalculate capacity, notify requester

rejectGuestRequest
Callable HTTPS function
Role: admin only
Action: reject guest, notify requester

sendAttendanceRequest
Callable HTTPS function
Role: admin only
Action: notify users linked to active members

sendAttendanceReminder
Callable HTTPS function
Role: admin only
Action: notify users with missing member responses

sendPendingGuestReminder
Callable HTTPS function
Role: admin only
Action: notify users with pending guest requests

sendOperationalUpdate
Callable HTTPS function
Role: admin or assigned helper
Action: notify attending users only

updateParkedBusLocation
Callable HTTPS function or secured Firestore write
Role: admin or assigned helper
Action: update parked bus pin and optionally notify attending users

assignEventHelper
Callable HTTPS function
Role: admin only
Action: assign helper to event

10. Notification Implementation

Store FCM tokens in:

```
users/{userId}/fcmTokens/{tokenId}
```


When the app starts:

1. Request notification permission.
2. Get FCM token.
3. Save/update token with platform and timestamp.
4. Listen for token refresh.

Notification payloads should include:

```
{
  "eventId": "event-id",
  "type": "attendance_request",
  "screen": "event_detail"
}
```

Notification categories:

```
attendance_request
attendance_reminder
pending_guest_reminder
guest_approved
guest_rejected
capacity_alert
operational_update
```

Operational updates must only target users with at least one `attending` member response for that event, plus admins/helpers.

11. Capacity Calculation Implementation

Create a shared backend helper:

```
async function recalculateEventCapacity(eventId: string): Promise<void>
```

Steps:

1. Load event.
2. Count member responses where `status == attending`.
3. Count guest requests where `status == approved`.
4. Count guest requests where `status == pending`.
5. Calculate approved and potential totals.
6. Derive `capacityStatus`.
7. Set `pendingGuestRisk`.
8. Update event summary fields.

9. If capacity status changed to near/at/over, send admin capacity alert.
10. If pending guest risk changed to true, send admin risk alert.

Persist these fields on the event for fast admin display:

```
{
  "capacityConfirmedMemberSeats": 42,
  "capacityApprovedGuestSeats": 5,
  "capacityPendingGuestSeats": 3,
  "capacityApprovedTotal": 47,
  "capacityPotentialTotal": 50,
  "capacityStatus": "near",
  "pendingGuestRisk": false,
  "capacityLastCalculatedAt": "timestamp"
}
```

12. Maps and Location Implementation

For route stops:

- Store `lat`, `lng`, address text, label, and notes.
- Allow manual coordinate entry or map-based selection in admin/helper UI.
- Use a maps launcher package to open locations externally.

For parked-bus location:

1. Admin/helper opens assigned event.
2. Taps **Set parked bus location**.
3. App requests foreground location permission.
4. App gets current GPS coordinates.
5. Admin/helper confirms or adjusts label/notes.
6. App saves location to event.
7. App offers to send operational update.

Avoid continuous tracking in MVP. Only store explicit pinned locations to reduce privacy risk and battery usage.

13. Firestore Security Rules

Implement security rules to enforce:

- users read only their own account and linked member information
- users can only respond for linked members
- users can only update before cutoff
- helpers can only update assigned event operational fields
- admins can manage everything
- guest approval is admin-only

- notification sending is function-only

Use custom claims:

```
{
  "admin": true,
  "helper": true
}
```

A user can be both admin and helper, but admin permission takes precedence.

14. Indexes

Create Firestore indexes for:

```
memberUserLinks: userId + status
memberUserLinks: memberId + status
members: status + displayName
events: status + eventDate
events: eventDate
memberResponses collection group: memberId + status + eventDate
memberResponses collection group: status + eventDate
guestRequests collection group: requestedByUserId + status
guestRequests collection group: status + createdAt
notifications: eventId + createdAt
```

15. Audit Trail

Create `auditLogs/{auditLogId}` for important admin/helper actions.

```
{
  "actorUserId": "admin-or-helper-id",
  "action": "approve_guest_request",
  "entityType": "guestRequest",
  "entityPath": "events/eventId/guestRequests/requestId",
  "before": {},
  "after": {},
  "createdAt": "timestamp"
}
```

Audit these actions:

- member created/edited/deactivated
- member linked/unlinked to user

- event created/edited/cancelled
- route stop changed
- parked-bus location changed
- guest approved/rejected
- admin override applied
- attendance reminders sent
- operational update sent
- helper assigned/removed

16. Admin Historical Reporting

Build history screen filters:

```
member
start date
end date
event title search
attendance status
```

Initial MVP reports:

- member attendance history
- event attendance list
- guest history by event
- members who have not attended within a selected date range

Export can be added later, but the UI should structure data so CSV export is straightforward.

17. Testing Plan

Unit tests:

- capacity calculation
- cutoff enforcement
- role permission helpers
- reminder target selection
- guest approval/rejection state changes

Integration tests:

- phone login mocked in Firebase emulator
- user submits member response
- user submits guest request
- admin approves guest
- helper updates parked bus location
- admin sends reminder

Manual acceptance tests:

- father account can respond for himself and two children
- guest request remains pending until admin approval
- approved guest affects capacity
- pending guest risk appears before approval
- route change notifies attending users only
- non-attending users do not receive operational updates
- helper cannot access full admin area
- admin can view past attendance by member

Device compatibility tests:

- one older supported iPhone running iOS 13 or closest available simulator/device
- one recent iPhone
- one older Android device or emulator targeting Android 7/API 24+
- one recent Android device

18. Release and Operations

Development release:

- Firebase dev project
- TestFlight internal testing
- Google Play internal testing
- seeded test members and events

Production release:

- Firebase prod project
- Apple App Store release
- Google Play closed or production release
- production member import
- admin training session

Operational runbook should cover:

- adding/editing members
- linking a user to multiple members
- creating an event
- adding route stops
- sending attendance request
- approving guests
- handling over-capacity
- assigning helpers
- setting parked-bus location
- querying past attendance
- disabling a lost/changed phone number

19. Data Migration and Backups

Before launch:

- import all active members
- validate phone numbers
- assign initial admins
- create sample event
- test notification delivery

After launch:

- schedule regular Firestore exports
- keep admin audit logs
- document manual recovery steps for accidental event/member changes

20. MVP Definition of Done

The MVP is complete when:

- users can log in with phone OTP
- users can request their own and child/dependent member links during signup
- admins can manage around 60 members
- admins can approve or manually create links from one user to multiple members
- admins can create an event with one bus and capacity
- admins can configure route stops
- users can confirm attendance for linked members
- members can select pickup and return stops
- users can request named guests with pickup selection
- admins can approve/reject guests
- capacity warnings work
- attendance reminders work
- helpers can update operational event details
- operational notifications go to attending users only
- admins can view upcoming and past events
- admins can query historical attendance by member
- the app is tested on supported older iOS and Android versions

Risks and Mitigations

Risk	Impact	Mitigation
Some members have very old or non-smartphones	Users may not be able to install or use the app	Keep admin manual entry and phone/SMS outside the app as the fallback process.

Risk	Impact	Mitigation
Phone number changes	A user may lose access or be linked to the wrong member records	Admins must be able to update user/member links and deactivate old links.
Push notifications are missed	Members may fail to respond	Admin attendance board shows still-to-confirm users, and admins can send repeat reminders or manually override.
Bus overbooking	Operational issue on match day	Capacity alerts, pending guest risk warnings, and admin-only guest approval reduce risk.
Helpers accidentally change critical data	Route or event confusion	Helper role is scoped to assigned events and operational updates only.
Location privacy concerns	Users may object to tracking	MVP uses explicit pinned locations only, not live tracking.
Firestore rules are too permissive	Data leak or unauthorised changes	Implement security rules, Cloud Function checks, and emulator tests before production.
Admin workload remains high	App may not deliver value	Post-launch metrics track manual overrides, reminders, and attendance response rates.

Open Decisions for Build Start

These do not block the architecture, but should be confirmed before contractors start implementation:

- Exact app name and branding.
- Whether the app will be published publicly in app stores or distributed privately/closed testing only.
- Who the first production admins are.
- Whether guest names should be retained permanently or removed/anonymised after a retention period.
- Whether attendance history should be exportable in the MVP or added immediately after launch.
- Whether map stop entry should use manual coordinates first or include address search/geocoding in the first release.

Milestones

Milestone 1 — Product Finalisation and Setup

Deliverables:

- final MVP scope agreed
- Firebase dev and prod projects created

- Flutter project created
- repository structure created
- basic CI checks configured
- Firebase Auth, Firestore, FCM, Functions, and Crashlytics enabled
- initial Firestore rules and indexes scaffolded

Acceptance criteria:

- app builds locally for iOS and Android
- app connects to Firebase dev project
- contractor team can run the app from documented setup steps

Milestone 2 — Authentication and User Foundation

Deliverables:

- phone number login with one-time code
- user profile creation/loading
- role loading for user/helper/admin
- FCM token registration
- basic navigation by role

Acceptance criteria:

- user can log in with phone OTP
- app stores and refreshes FCM token
- admin/helper/user roles route to the correct screens
- inactive users are blocked with a clear message

Milestone 3 — Member and Representation Management

Deliverables:

- admin member list
- create/edit/deactivate member
- import script for initial member list
- user-to-member linking
- support for one user representing their own member record and multiple child/dependent members

Acceptance criteria:

- user can request during signup to represent themselves and two child members
- admin can approve all three member links to one user phone account
- after approval, that user can see all three represented members after login
- inactive members do not appear in normal attendance flows

Milestone 4 — Event and Route Management

Deliverables:

- admin event create/edit screen
- capacity setting
- optional cutoff date/time
- event status handling
- route stop editor
- outbound pickup, event drop-off, event pickup, and return drop-off stop types
- map coordinate storage

Acceptance criteria:

- admin can create an event with one bus and a maximum capacity
- admin can configure several outbound pickups and return drop-offs
- admin/helper can edit route stops on the day
- users can open route stop locations in Apple Maps or Google Maps

Milestone 5 — Member Attendance Flow

Deliverables:

- current/upcoming events list for users
- event detail screen
- linked member attendance form
- pickup and return drop-off selection for attending members
- cutoff enforcement
- admin override support

Acceptance criteria:

- user can mark each linked member as attending or not attending
- attending members must select valid route stops where required
- attendance is confirmed immediately without admin approval
- after cutoff, user changes are blocked
- admin override still works after cutoff

Milestone 6 — Guest Requests and Admin Approval

Deliverables:

- named guest request form
- guest initial pickup selection
- pending/approved/rejected states
- admin guest approval queue
- guest approval/rejection notifications

Acceptance criteria:

- user can submit named guest requests before cutoff
- guests start as pending and do not immediately count as approved capacity
- admin can approve or reject each guest
- approved guests count against capacity
- user is notified when a guest is approved or rejected

Milestone 7 — Capacity, Reminders, and Notifications

Deliverables:

- capacity calculation Cloud Function
- near/at/over capacity status
- pending guest risk indicator
- admin capacity alerts
- attendance request notification
- attendance reminder notification
- pending guest reminder notification
- notification audit records

Acceptance criteria:

- capacity totals update after member response and guest approval changes
- admins are alerted when capacity is near, at, or over the limit
- admins are alerted when pending guests could exceed capacity
- reminders only target the correct users
- notification history is visible for troubleshooting

Milestone 8 — Helper Operations and Parked-Bus Location

Deliverables:

- helper assignment per event
- helper assigned-events screen
- parked-bus location pin update
- operational event update notifications
- helper-scoped permissions

Acceptance criteria:

- admin can assign multiple helpers to an event
- helper can update parked-bus location and notes for assigned events
- helper cannot manage members, approve guests, or send attendance reminders
- operational updates go only to users marked as attending, plus admins/helpers

Milestone 9 — Admin Attendance Board and History

Deliverables:

- attendance board grouped by attending, not attending, and still to confirm
- guest request views grouped by pending, approved, and rejected
- route selection visibility
- past event access for admins
- historical attendance query by member, event, and date range

Acceptance criteria:

- admin can view all event attendance states clearly
- admin can identify who has not responded
- admin can view historical attendance for a selected member
- normal users only see current/upcoming events
- admins can access past events

Milestone 10 — Security, Testing, and Release

Deliverables:

- complete Firestore security rules
- Cloud Function permission checks
- audit logging
- Crashlytics integration
- test coverage for core business rules
- compatibility testing on older supported iOS and Android versions
- TestFlight and Google Play internal testing builds
- production release checklist
- admin runbook

Acceptance criteria:

- users cannot respond for unlinked members
- helpers cannot access admin-only functions
- cutoff and guest approval rules are enforced server-side
- core workflows pass manual acceptance testing
- app is stable on supported older devices
- admins are trained using the runbook

Gathering Results

Success Criteria After Launch

The system should be evaluated against product, operational, and technical measures.

Product Measures

Track whether the app solves the real bus-management problem:

```
attendance response rate per event
percentage of members confirmed before cutoff
number of reminder notifications needed per event
number of manual admin overrides per event
guest requests submitted per event
guest requests approved/rejected per event
number of users representing multiple members
```

Target MVP outcomes:

- at least 90% of member attendance responses submitted before cutoff after the first few events
- fewer manual phone/message confirmations over time
- admins can produce an accurate attendance list without using spreadsheets as the primary source
- guest approvals and capacity decisions are visible in one place

Operational Measures

Track whether match-day operation is smoother:

```
number of route changes per event
number of operational notifications sent
number of parked-bus location views
number of users opening map links
admin-reported incidents of people at the wrong pickup/drop-off point
```

Target MVP outcomes:

- attending users can see current route/stop information without asking admins directly
- parked-bus location can be updated quickly by an admin/helper
- operational updates reach only attending users
- helpers can manage match-day updates without needing full admin access

Capacity and Guest Management Measures

Track whether capacity control is working:

```
events near capacity
events at capacity
events over capacity
pending guest risk alerts
```

approved guest count versus rejected guest count
capacity alert acknowledgement by admins

Target MVP outcomes:

- admins receive capacity warnings before the bus is overbooked
- approved and pending guest seats are clearly separated
- admins can make guest approval decisions using current seat totals

Technical Measures

Track app health and reliability:

crash-free sessions
push notification delivery failures
Cloud Function errors
Firestore permission errors
authentication failures
slow screen loads
failed location permission flows

Target MVP outcomes:

- app remains stable on supported older iOS and Android versions
- no repeated Cloud Function failures during event workflows
- no security-rule gaps allowing unauthorised attendance changes
- push notifications are reliable enough that admins do not need to duplicate every reminder manually

Post-Launch Review Process

After each of the first 3 live events:

1. Admin exports or reviews attendance results.
2. Admin records any manual workarounds needed.
3. Admin checks whether capacity and guest approvals were accurate.
4. Helpers confirm whether parked-bus and route updates were easy to manage.
5. Users report any login, notification, or route confusion.
6. Contractor team fixes high-priority issues before the next event.

After 5–10 events, review whether to add:

- CSV/PDF export
- multiple buses per event
- SMS fallback integration
- richer pickup/drop-off reporting

- better map search/geocoding
- waitlist handling
- payment collection

MVP Acceptance Checklist

The MVP should be considered successful when:

- admins no longer rely on manual spreadsheets as the main attendance source
- users can confirm attendance for represented members without admin help
- guest seats cannot bypass admin approval
- capacity warnings prevent accidental overbooking
- route and parked-bus updates are visible to attending users
- historical attendance is queryable by member
- the app works acceptably on the oldest supported phones used by the group

Need Professional Help in Developing Your Architecture?

Please contact me at sammuti.com :)

Gathering Results

To be completed after requirements and method are confirmed.

Need Professional Help in Developing Your Architecture?

Please contact me at sammuti.com :)